論文内容の要旨

Optimizing Annotation Guideline Refinements for LLM-Based Annotation

（LLM によるアノテーションに向けたガイドライン改善の最適化）

Hengyu Zhou

1. Introduction

Text annotation such as biomedical named entity recognition (NER) follows annotation guidelines, the concrete labeling rules for human annotators. The human annotation process is slow and expensive, so transferring the guidelines to an LLM is an attractive alternative. But no written guidelines can state every case, and they often fail to clearly describe an entity's scope, gray-zone cases, and span boundaries. Human annotators can usually handle this incompleteness by their implicit knowledge on biomedicine and annotation. An LLM can also utilize the guideline but no such implicit knowledge on annotation. Every convention a human annotator would internalize therefore has to be written into the guideline itself, so that the LLM can apply it.

Prior work closes this gap by iteratively updating the guideline through the workflow of extracting the missing conventions from gold annotations and reflecting the missing conventions (Figure 1, adapted from [1]). Specifically, the LLM annotates a ten-document development set under the current guideline, and its predictions are scored against the gold annotations using F1. The disagreements are grouped, and the largest group becomes a target for refinements: it is explained as a linguistic pattern, condensed into one rule, and written back into the guideline for the next round. The loop stops when development F1 reaches 0.9 or a round fails to improve it. The experiments using NCBI Disease with GPT-5, measured on a validation set of 100 documents showed the refinement of the F1 score 0.76 compared with the simple prompting (0.46) and simple prompting with the original guideline (0.73)[1]. Although adding the original guideline to the prompt improved quite a lot (+0.27), improvements with the iterative refinements are limited (+0.03). In this research, we analyzed the detailed cause of the limited refinements to achieve further improvement.

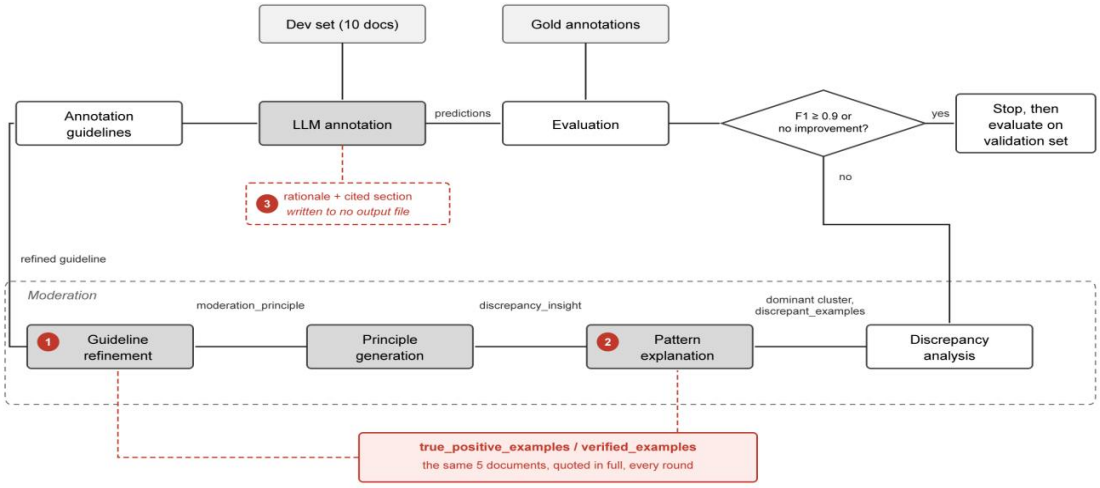

Figure 1. One round of iterative guideline moderation as implemented. Markers 1 to 3 locate points 1, 2 and 3 of Table 1.

2. Methodology

To optimize guideline refinement for LLM-Based annotation by maximizing what moderation can contribute, we first examined the released implementation stage by stage to locate what limits the current gain. Table 1 lists the six points found, with the evidence for each.

#	Point	Evidence
1	Verified examples, meant only to prevent regression, are copied into the guideline instead (i.e. hard coding).	38 of the 52 lines round 1 adds to refined guidelines come from the five verified example documents to ask the model not to flip them.
2	The verified examples do not change with the error type being analyzed	Provided reference verified examples are identical across rounds, while the error type varies. In round 3, the largest discrepancy cluster is a boundary mismatch, and provided verified examples have no such evidence.
3	The rationale each annotation carries is not fully utilized	Every annotation names a rationale and the section it applied; neither is recorded, so neither can be analyzed
4	Ten development documents do not cover the error types the guideline is judged on	Gold Modifier predicted as nothing: 21% of validation-set disagreements, the largest discrepancy cluster on the validation set, but 0% of the development set's.
5	Annotating the same documents twice does not give the same result	Two annotations of the same ten documents can return 59 and 56 correct annotations; over twenty annotations, development F1 ranged 0.739 to 0.859
6	One group is moderated per round, so each repair causes regressions elsewhere	Each round repairs its target but adds errors of other types; the total falls only from 18 to 11 over three rounds (Figure 2)

Table 1. Six points at which the released implementation could be improved, with the evidence for each.

To select improvement directions, we grouped the six points along the following three axes: 1. How much the loop extracts from the development set it is given (points 2, 3, 6); 2. How much of what it extracts generalizes to the validation set (points 1, 4); 3. How reliably either can be measured (point 5). All three are quantified in this reproduction. On the first,

over three rounds the loop targets 12 disagreements and the total falls by only 7. On the second, development F1 rises 0.11 against 0.01 on the validation set. On the third, repeated annotation of the same ten documents under one unchanged guideline gives development F1 anywhere between 0.739 and 0.859.

3. Current Results

One point from each axis is taken forward: points 1, 6 and 5.

First, the mechanism that limits regression also limits the generalization of unseen data. Each refinement iteration is given the annotations the current guideline already gets right (TPs), together with the full text of the development documents TPs were taken from; the LLM is then told to preserve them so that the regression is limited. That source text is copied into the prompt with nothing to mark it as reference only rather than material that could be copied into the refined guideline, so the cheapest way to preserve an example is to restate it: the documents' own wording is written into the guideline as rules. Of the 52 lines round 1 adds, 38 can be traced back to the development documents, this rate is defined as Copied Share. The guideline stops being a specification of the entity types and becomes a hard coding record of the documents it was tuned on, which is why the development F1 gain does not generalize to the unseen validation set.

We ran two interventions against this. The first removed the verified examples; the second supplied them but required every rule taken from them to be stated in abstract terms and every illustrative example to be invented. Both stopped the copying, but neither reached the stopping threshold of F1 $\geq$ 0.9 (Table 2). Designs that limit regression without putting the source documents in the prompt are being tested.

Condition	Development F1	Threshold reached	Validation F1	Copied Share
Verified examples supplied	0.797 ± 0.015 → 0.909 ± 0.015	Yes	0.779→0.789	38/52 = 73%
Verified examples removed	0.797 ± 0.015 → 0.837 ± 0.015	No	0.779→0.771	0/32 = 0%
Verified examples supplied, copying forbidden	0.823 ± 0.015 → 0.833 ± 0.015	No	0.779→0.784	0/25 = 0%

Table 2. Development F1, Validation F1 and Copied Share under three conditions, NCBI Disease with GPT-5.4, ten development documents.

Second, moderating one group at a time causes regressions in the others. Each round targets a single {gold type, predicted type} group. The total disagreement count falls only from 18 to 11 over three rounds (Figure 2), because each round adds errors of a type it was not targeting. Round 1 cuts its target, 'gold SpecificDisease predicted as DiseaseClass', from 6 to 2, but adds a 'missed DiseaseClass' and a 'SpecificDisease boundary' error. Round 2 halves those new boundary errors but pushes the previous target back up to 3. To solve this problem, targeting several groups within one rewrite, rather than one at a time, would let each round be checked against more than one discrepancy cluster before it is accepted. A rewrite that moderates the top k groups at once is being designed, and the experiment is scheduled.

(a) Iter 0 (G0)

Gold ↓ / Pred →	C	D	M	S	O
C	0	0	0	0	1
D	0	0	0	1	2
M	0	0	0	0	0
S	0	6	2	2	3
O	0	0	0	1	0

(b) Iter 1 (G1)

Gold ↓ / Pred →	C	D	M	S	O
C	0	0	0	0	1
D	0	0	0	1	3
M	0	0	0	0	0
S	0	2	2	3	3
O	0	0	0	0	0

(c) Iter 2 (G2)

Gold ↓ / Pred →	C	D	M	S	O
C	0	0	0	0	1
D	0	0	0	1	3
M	0	0	0	0	0
S	0	3	2	1	2
O	0	0	0	0	0

(d) Iter 3 (G3)

Gold ↓ / Pred →	C	D	M	S	O
C	0	0	0	0	1
D	0	0	0	0	2
M	0	0	0	0	0
S	0	2	2	1	3
O	0	0	0	0	0

Figure 2. Discrepancy counts by {gold type, predicted type} group for the initial guideline and after each of the three moderation rounds, NCBI Disease with GPT-5.4, ten development documents. Gold type runs down, predicted type across; the diagonal holds span boundary mismatches[1].

Third, a single annotation is not a stable measurement. We annotated the same ten documents 20 times in parallel; the resulting F1 scores ranged between 0.739 and 0.859. The refinement loop keeps rewriting when F1 rises and stops when it does not, and the F1 improvement gained from refinement loops is smaller than that spread, so the decision can turn on noise alone. We found the randomness cannot be switched off: we ran ten parallel annotations with DeepSeek-V4, a reasoning LLM, at temperature 0 and temperature 1; the deviation was 0.0261 and 0.0147 respectively, because for reasoning models the reasoning process is sampled independently of the answer. The randomness can be averaged down: we annotated datasets of size 10, 20 and 30, and the deviation fell from 0.0150 at ten documents to 0.0092 at thirty. Enlarging the development set is therefore effective against the noise, and running the loop on a larger and coverage-balanced set is under way.

4. Discussion and Future Work

The development gain does not transfer. Of the three conditions in Table 2, only the verified-examples-supplied condition reached the development-F1 stopping threshold, and on the validation set it scored no better than the two conditions that never reached it. The copying cannot simply be removed either: the verified examples are the only thing stopping a rewrite from breaking annotations that already pass, and without them the loop stops improving after one or two rounds. What lets the guideline converge is what ties it to the ten development documents.

Three points remain evidenced but untested: the verified examples do not change with the error type being moderated (point 2), the rationale each annotation carries is recorded nowhere and so cannot guide the rewrite (point 3), and ten development documents do not cover the error type behind the largest share of validation disagreements (point 4). A rewrite that moderates the top k groups at once is already scheduled for point 6.

References

[1] Refining and Reusing Annotation Guidelines for LLM Annotation (Kim et al., ACL 2026).